

Paris Gun: Computational Analysis of Projectile in Non-Inertial Reference Frame

Junyu Hu,^{*} Songhang Li,[†] and Kowalski Sai[‡]
Franklin & Marshall College Department of Physics
(Dated: March 14, 2026)

The Paris gun was built by German in World War I as a terror weapon. We are interested in the physics behind the high speed motion of an object and its projectile under the various external forces. In this lab, we simulated the projectile motion of the Paris Gun and analyzed the effect of the quadratic drag force, gravity, Coriolis force, and centrifugal force in the non-inertial frame using Runge-Kutta and Euler's method. By implementing the altitude-dependent air density function, we also simulated the effect of the change of the air density in a long distance movement and discussed the effect scale of air density and Coriolis force. We also compared the results of RK4 and Euler's methods.

1. INTRODUCTION

The Paris Gun was a type of German long-range gun built for bombarding the Paris during World War I. They were in service only for short period of time because of their low accuracy and small payload. However, Paris Gun provides rich physics context relating to the non-inertial reference frame and the effect of air density in long distance motion of the large scale object. In the following sections, we analyzed the effects of the quadratic drag force, gravity, Coriolis force, and centrifugal force in the rotational frame, defined by x , y , and z hereafter. Using the Runge-Kutta method and Euler's method to numerically solve the equations of motion, we simulate the projectile and found that there is not significant difference between these two method.

2. THEORY

2.1. Equation of Motion

Newton's Second Law no longer works in a rotational (non-inertial) frame, so we have to add some "correction terms" to make the system mathematically follow the Newton's Second Law just like in the inertial reference frame. In a rotating frame, there exists Coriolis and centrifugal forces in addition to the gravity and drag. To make the projectile more precise and closer to the real circumstance, we also integrated the altitude-dependent air density function in our program to simulate the effect of the change of the air density. We represented $\vec{\Omega}$ in terms of the xyz -basis: $\vec{\Omega} = -\Omega \sin \theta \hat{x} + \Omega \cos \theta \hat{y} + 0 \hat{z}$.

The force equations are (calculation details of each force are included in the Appendix):

$$\vec{F}_{cor} = 2m\vec{v} \times \vec{\Omega}$$

$$\vec{F}_{cf} = m(\vec{\Omega} \times \vec{r}) \times \vec{\Omega}$$

$$\vec{F}_{drag} = -cv\vec{v}$$

$$\vec{F}_{net} = m\vec{g} + \vec{F}_{cor} + \vec{F}_{cf} + \vec{F}_{drag}$$

$$\vec{a} = \begin{bmatrix} 0 \\ -g \\ 0 \end{bmatrix} + \begin{bmatrix} -2\Omega v_z \cos \theta - cv_x \\ -2\Omega v_z \sin \theta \\ 2\Omega v_x \cos \theta + 2\Omega v_y \sin \theta \end{bmatrix} + \begin{bmatrix} \Omega^2 \cos \theta (y \sin \theta + x \cos \theta) \\ \Omega^2 \sin \theta (y \sin \theta + x \cos \theta) \\ \Omega^2 z \end{bmatrix} + \begin{bmatrix} -\frac{cv}{m} v_x \\ -\frac{cv}{m} v_y \\ -\frac{cv}{m} v_z \end{bmatrix}$$

^{*}Electronic address: jhu2@fandm.edu

[†]Electronic address: sli2@fandm.edu

[‡]Electronic address: ysai@fandm.edu

, where m is the mass of the projectile, $\vec{v}$ is its velocity and $\vec{\Omega}$ is the angular velocity of the Earth's rotation.

Collecting all terms, we have the following system of equations consisting of second-order coupled partial differential equations:

$$\begin{aligned}\frac{d}{dx}v_x &= -2\Omega v_z \cos \theta - \frac{cv}{m}v_x + \Omega^2 \cos \theta (y \sin \theta + x \cos \theta) \\ \frac{d}{dx}v_y &= -g - 2\Omega v_z \sin \theta - \frac{cv}{m}v_y + \Omega^2 \sin \theta (y \sin \theta + x \cos \theta) \\ \frac{d}{dt}v_z &= 2\Omega v_x \cos \theta + 2\Omega v_y \sin \theta - \frac{cv}{m}v_z + \Omega^2 z\end{aligned}\tag{1}$$

However, it is unlikely to find an analytical solution to this system. Therefore, we used numerical methods to solve this system of equations. In the next section, we will discuss how we implemented the Runge Kutta and Euler Methods in the program.

3. COMPUTATIONAL METHODS [1]

3.1. Euler Method

Suppose we have a generic function $f(t)$ and its derivative $\frac{df}{dt} = \dot{f}$, where the derivative $\frac{df}{dt} = \dot{f}$ depends on both t and $f(t)$ itself. So we define a derivative function $\dot{f}(f, t)$ to highlight the dependence between the function and its derivative at a certain time step, i.e., the differential equation. In the second-law context, assuming $f(t)$ is the velocity, so $\dot{f}(t, f)$ is acceleration. After looping around the calculation (use while loop to repeat step 3 and 4), we obtained an array of velocity. Then we used this trick again to get an array of position that plot the projectile.

Euler's method allows us to approximate the solution $f(t)$ from $\dot{f}(f, t)$ and $f(t=0)$ using the following steps:

1. Start with the initial value $f(0) = f_0$.
2. Calculate the derivative $\dot{f}(f, t)$ by plugging in $f = f_0$ and $t = 0$.
3. Approximate the new function value after a short time Δt $f(t + \Delta t) = f(t) + \dot{f}\Delta t$.
4. Calculate the new derivative $\dot{f}$ by $\frac{f(t+\Delta t)}{t+\Delta t}$ based on step 3 and plug in to next loop of $f(t + \Delta t) = f(t) + \dot{f}\Delta t$.
5. Repeat steps 3 and 4 many times to find the value of $f(t)$ in future time t .

However, Euler method could cause larger divergence in large time scale because the estimation of the next step is based on the previous step and the discrepancy adds up. Therefore, we used the better Runge Kutta 4 method to approximate the projectile.

3.2. Runge Kutta Method

According to the Runge Kutta 4 method, the current step can be estimated using:

$$\begin{aligned}f_i &= f_{i-1} + \Delta f & \Delta f &= \frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} \\ k_1 &= \dot{f}(f_{i-1}, t_{i-1}) \Delta t \\ k_2 &= \dot{f}\left(f_{i-1} + \frac{k_1}{2}, t_{i-1} + \frac{\Delta t}{2}\right) \Delta t \\ k_3 &= \dot{f}\left(f_{i-1} + \frac{k_2}{2}, t_{i-1} + \frac{\Delta t}{2}\right) \Delta t \\ k_4 &= \dot{f}(f_{i-1} + k_3, t_{i-1} + \Delta t) \Delta t \\ \Delta f_{\text{RK4}} &= \frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4\end{aligned}$$

where

1. k_1 is simply the value of $\Delta f = \dot{f}dt$ as we used for the Euler method: calculate $\dot{f}(f, t)$ by plugging in t_{i-1} and f_{i-1}
2. k_2 estimates $\dot{f}$ at the halfway point between t_{i-1} and t_i by taking $t = t_{i-1} + \Delta t/2$ and $f = f_{i-1} + \Delta f/2$, where Δf is taken to be k_1
3. k_3 estimates $\dot{f}$ at the halfway point between t_{i-1} and t_i by taking $t = t_{i-1} + \Delta t/2$ and $f = f_{i-1} + \Delta f/2$, but here Δf is taken to be k_2
4. k_4 estimates $\dot{f}$ at t_i by plugging in $t = t_{i-1} + \Delta t$ and $f = f_{i-1} + \Delta f$, where in this case Δf is taken to be k_3

4. RESULTS AND ANALYSIS

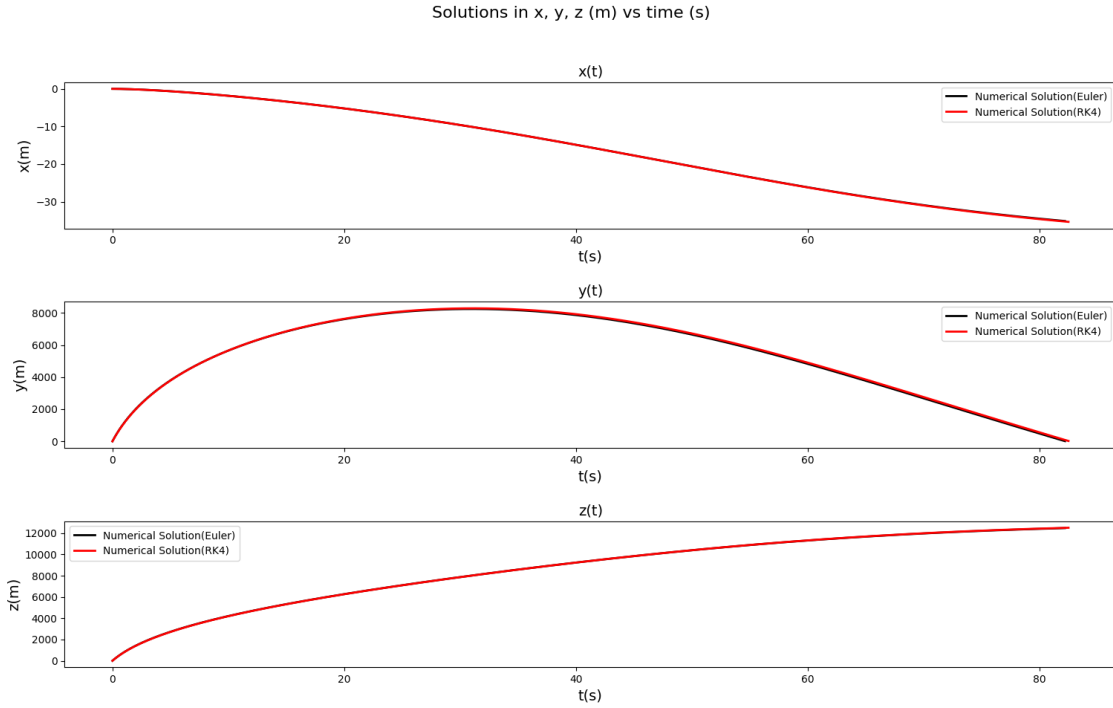


FIG. 1: Comparison of Euler and RK4 method with all effects, x, y and z are all in Inertial frame with air drag, $x+$ -direction is southward, $y+$ -direction is upward and $z+$ -direction is westward in Figure.2.

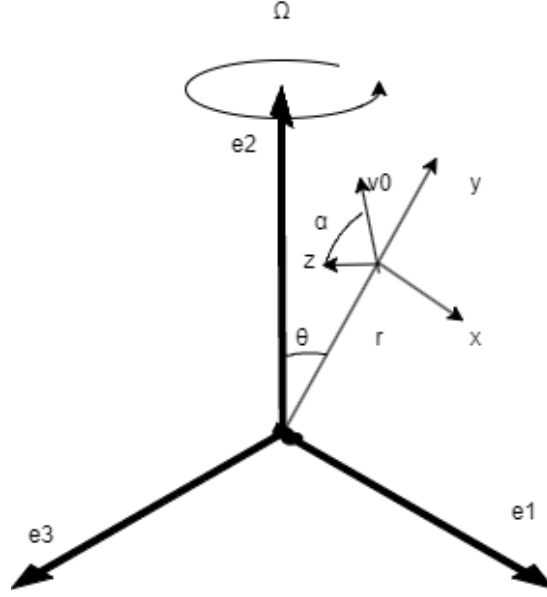


FIG. 2: The rotational reference frame. xyz coordinate is the frame we used in the code to plot the path, e -basis coordinate is for analyzing the Coriolis and centrifugal forces. The Gun shoots Mass of the shell 106 kg, initial velocity $v_0 = 1640 \text{ m/s}$ with elevation $\alpha = 55^\circ$, The launch site is located at 49° north latitude then $\theta = 90^\circ - 49^\circ = 41^\circ$. The air Drag coefficient $c = 0.1$.

The projectile's motion was simulated under three distinct conditions: an inertial frame with air drag (Figure 1), an inertial frame without air drag (Figure 3), and a non-inertial (rotating) frame with air drag (Figure 4). Each simulation generated 825 data points, with representative subsets (10 data points per method) are shown in (Table. I) of Appendix A where x corresponds to south, y corresponds to up and z corresponds to west.

4.1. Non-Inertial Frame Effects: Coriolis and Centrifugal Forces

From the Figure. 3 compared with Figure. 1, the Coriolis force induced a southward deflection (x -axis) about 30 m. This aligns with theoretical predictions from Equation.1, where $\vec{\Omega}$ cross product with the projectiles's velocity to generate the component in the third direction. In the code, We set the force in Non-Inertial frame in the resultant part of the code to 0, the part of code in z -axis shows as follows:

```
# dv_cor is the Coriolis force, dv_cf Centrifugal Force, dv_drag is air-drag
dv_cor    = 0
dv_drag   = c_drag * speed * v_3
dv_cf     = 0
```

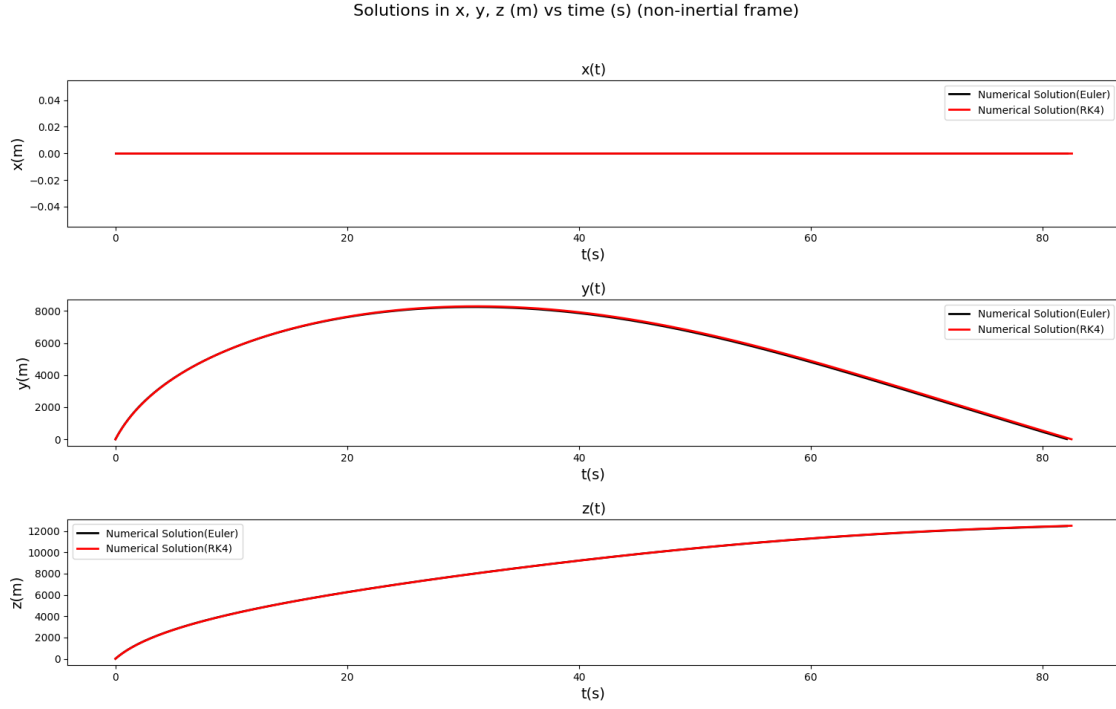


FIG. 3: Comparison of Euler and RK4 method in a non-inertial frame, x , y and z are all in Non -Inertial frame with air drag, $x+$ -direction is southward, $y+$ -direction is upward and $z+$ -direction is westward in Figure.2.

4.2. Air Density Variation and Drag Effects

From the Figure. 4 compared with Figure. 1, with altitude-dependent (y -axis) air density, the maximum horizontal range decreased about $130000m$. The quadratic drag force caused velocity decreased at higher altitudes with the relation of air density. In same way, We set the force due to air-drag in the resultant part of the code to 0, the part of code in z -axis shows as follows:

```
# dv_cor is the Coriolis force, dv_cf Centrifugal Force, dv_drag is air-drag
dv_cor    = 2 * omga*(v_1 * np.cos(theta) + v_2 * np.sin(theta))
dv_drag   = 0
dv_cf     = omga**2 * z
```

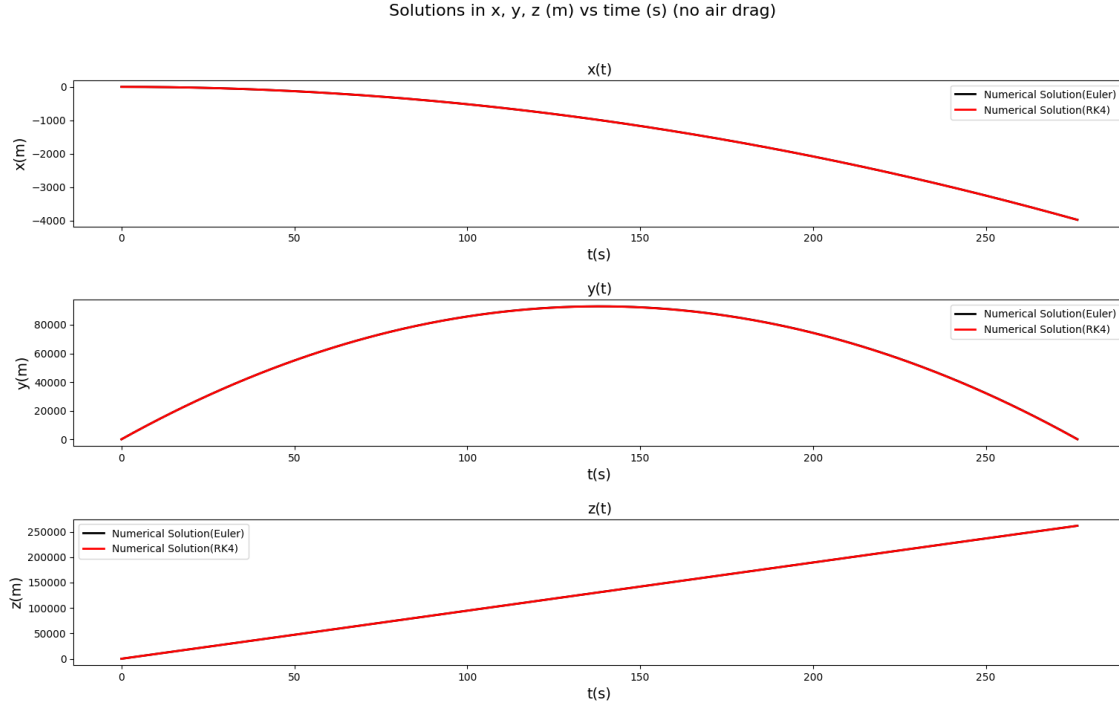


FIG. 4: Comparison of Euler and RK4 method, x , y and z are all in Intertial frame without air drag, $x+$ -direction is southward, $y+$ -direction is upward and $z+$ -direction is westward in Figure.2 .

4.3. Numerical Method Comparison

The RK4 method produced smoother and more stable trajectories than Euler, especially at high velocities. Due to we don't have analytical solution (real data), the difference would be treated as uncertainty of analytical solution (estimation of trajectory). The advantages of these two methods are different: Tests with varying step sizes (dt) demonstrated that Euler's results diverged rapidly for $dt \geq 0.5s$, leading to unphysical oscillations in altitude and range. In contrast, RK4 kept convergence even at $dt \in (0.5, 1)s$ with reduced resolution. The RK4 would spend more time for it required 4 more computational effort per step (in this lab with 8 for second order differential equation) than Euler, its ability to use larger step size without sacrificing accuracy made it more efficient overall for long-duration simulations.

5. CONCLUSIONS

In this lab, we simulated the long range projectile of the Paris Gun and analyzed the effect of different forces using Runge-Kutta and Euler's method.

According to the three graphs with the data in table, we conclude that quadratic drag and altitude-dependent air density reduced the Paris Gun's range about 65% compared to a vacuum model, demonstrating the significance of the drag in artillery simulations. Furthermore, the Coriolis force is non-negligible. The deflection ($1 - 2km$ southward and $500m$ westward) shows that Earth's rotation significantly influences the motion simulation of the system. The RK4 method outperformed Euler in stability and precision, particularly in the high-velocity, long-duration simulations.

[1] Ryan Trainor, *Ordinary differential equations and the euler method* (Spring 2024), PHY 226, Department of Physics and Astronomy, Franklin and Marshall College.

Appendix A: Data Table

TABLE I: Extracted some data of graph, due to the total number of data is huge, the data in the table shows only the key trends in 10 sets

$t(s)$	$x_{eulder}(m)$	$y_{eulder}(m)$	$z_{eulder}(m)$	$x_{RK4}(m)$	$y_{RK4}(m)$	$z_{RK4}(m)$
0	0	0	0	0	0	0
8.0	-1.3	5020.6	3674.0	-1.3	5017.6	3661.8
16.2	-3.8	7063.8	5564.3	-3.8	7079.2	5555.8
24.4	-7.0	8030.1	7009.9	-7.0	8060.8	7003.7
32.6	-10.9	8239.8	8246.0	-10.9	8284.1	8242.1
40.8	-15.3	7804.2	9339.0	-15.3	7860.9	9337.8
49.0	-20.0	6811.5	10291.6	-20.1	6878.8	10294.1
57.2	-24.7	5386.4	11084.9	-20.1	6878.8	810294.1
65.4	-28.9	3689.9	11705.5	-28.9	3768.2	11718.1
73.6	-32.4	1884.7	12159.4	-32.5	1962.7	12177.2
82.3	-35.2	-11.2	12485.8	-35.4	-0.7	12517.3

^aData with subscript is *eulder* the measurement of Euler's method, the other one is RK4 method

Appendix B: Calculation

Handwritten scan is attached in the end of the file.

$$\begin{aligned}
 F_{\text{cor}} &= 2m\vec{v} \times \vec{\Omega} = 2m \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ v_x & v_y & v_z \\ -\Omega \sin \theta & \Omega \cos \theta & 0 \end{vmatrix} = 2m ((0 - v_z \Omega \cos \theta) \hat{x} - (0 + \Omega v_z \sin \theta) \hat{y} + (v_x \Omega \cos \theta + v_y \Omega \sin \theta) \hat{z}) \\
 &= 2m \begin{pmatrix} -v_z \Omega \cos \theta \\ -\Omega v_z \sin \theta \\ v_x \Omega \cos \theta + v_y \Omega \sin \theta \end{pmatrix}
 \end{aligned}$$

$$F_{cf} = m(\vec{\Omega} \times \vec{r}) \times \vec{\Omega}:$$

$$\begin{aligned}
 \vec{\Omega} \times \vec{r} &= \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ -\Omega \sin \theta & \Omega \cos \theta & 0 \\ x & y & z \end{vmatrix} = \begin{pmatrix} \Omega \cos \theta z \\ \Omega \sin \theta z \\ -\Omega \sin \theta y - \Omega \cos \theta x \end{pmatrix} \\
 (\vec{\Omega} \times \vec{r}) \times \vec{\Omega} &= \begin{vmatrix} \hat{x} & \hat{y} & \hat{z} \\ \Omega \cos \theta z & \Omega \sin \theta z & -\Omega \sin \theta y - \Omega \cos \theta x \\ -\Omega \sin \theta & \Omega \cos \theta & 0 \end{vmatrix} = \begin{pmatrix} \Omega^2 \cos \theta (\sin \theta y + \cos \theta x) \\ \Omega^2 \sin \theta (\sin \theta y + \cos \theta x) \\ \Omega^2 \cos^2 \theta z + \Omega^2 \sin^2 \theta z \end{pmatrix}
 \end{aligned}$$

therefore, the Coriolis force is:

$$F_{\text{cor}} = m(\vec{\Omega} \times \vec{r}) \times \vec{\Omega} = \begin{pmatrix} \Omega^2 \cos \theta (\sin \theta y + \cos \theta x) \\ \Omega^2 \sin \theta (\sin \theta y + \cos \theta x) \\ \Omega^2 z \end{pmatrix}$$

Appendix C: source code

The Python code in three file for different function.
Here's the code in Main.py:

```
def rho(h):

    if h <= 11000:
        Tep = 288.1 - 0.00649 * h
        p = 101.29* (Tep / 288.08)**5.256
    elif h <= 25000:
        Tep = 216.6
        p = 22.65 * np.exp(1.73 - 0.000157 * h)
    else:
        Tep = 144.8 + 0.00299 * h
        p = 2.488* (Tep / 216.6)**(-11.388)

    density = p / (0.2869 * Tep)

    # make sure the density is not zero
    if density == 0:
        density = 1e-10 # 1e-10 is a small number

    return density

#####

# Define the velocity functions
# f_net = f_cor + f_drag + f_cf + f_gravity
# when we do the analysis for controlling variable, we will set some part of "force" = 0.

def v_xe(v_1, v_2, v_3, x, y, z, dt):

    c_drag = - k * d * rho(y)/m
    speed = np.sqrt(v_1**2 + v_2**2 + v_3**2)
    dv_cor = -2 * omga * v_3 * np.cos(theta)
    dv_drag = c_drag * speed * v_1
    dv_cf = omga**2 * np.cos(theta) * (np.cos(theta) * x + np.sin(theta) * y)
    return (dv_cor+dv_drag+dv_cf) * dt

def v_ye(v_1, v_2, v_3, x, y, z, dt):

    c_drag = - k * d * rho(y)/m
    speed = np.sqrt(v_1**2 + v_2**2 + v_3**2)

    dv_cor = 2 * omga * v_3 * np.sin(theta)
    dv_drag = c_drag * speed * v_2
    dv_cf = - omga**2 * np.sin(theta) * (np.cos(theta) * x + np.sin(theta) * y)

    return (dv_cor + dv_drag + dv_cf - g) * dt

def v_ze(v_1, v_2, v_3, x, y, z, dt):

    c_drag = - k * d * rho(y)/m
    speed = np.sqrt(v_1**2 + v_2**2 + v_3**2)

    dv_cor = 2 * omga*(v_1 * np.cos(theta) + v_2 * np.sin(theta))
    dv_drag = c_drag * speed * v_3
```



```

dv_cf      = omga**2 * z

return (dv_cor+dv_drag+dv_cf) * dt

#####
# RK4 method

def x_rk4(v_1, v_2, v_3, x, y, z, dt):

    def dvdt(v_1, v_2, v_3, x, y, z ):
        c_drag = - k * d * rho(y)/m
        speed = np.sqrt(v_1**2 + v_2**2 + v_3**2)
        dv_cor  = -2 * omga * v_3 * np.cos(theta)
        dv_drag  = c_drag * speed * v_1
        dv_cf    = omga**2 * np.cos(theta) * (np.cos(theta) * x + np.sin(theta) * y)
        return dv_cor + dv_drag + dv_cf

    k_v1 = dvdt(v_1, v_2, v_3, x, y, z) * dt
    k_v2 = dvdt(v_1 + k_v1/2, v_2, v_3, x, y, z) * dt
    k_v3 = dvdt(v_1 + k_v2/2, v_2, v_3, x, y, z) * dt
    k_v4 = dvdt(v_1 + k_v3, v_2, v_3, x, y, z) * dt

    k_x1 = v_1 * dt
    k_x2 = (v_1 + k_v1/2) * dt
    k_x3 = (v_1 + k_v2/2) * dt
    k_x4 = (v_1 + k_v3) * dt

    return (k_x1 + 2 * k_x2 + 2 * k_x3 + k_x4) / 6 , (k_v1 + 2 * k_v2 + 2 * k_v3 + k_v4)/ 6

def y_rk4(v_1, v_2, v_3, x, y, z, dt):

    def dvdt(v_1, v_2, v_3, x, y, z ):
        c_drag = - k * d * rho(y)/m
        speed = np.sqrt(v_1**2 + v_2**2 + v_3**2)
        dv_cor  = 2 * omga * v_3 * np.sin(theta)
        dv_drag  = c_drag * speed * v_2
        dv_cf    = - omga**2 * np.sin(theta) * (np.cos(theta) * x + np.sin(theta) * y)
        return dv_cor + dv_drag + dv_cf - g

    k_v1 = dvdt(v_1, v_2, v_3, x, y, z) * dt
    k_v2 = dvdt(v_1 , v_2+ k_v1/2, v_3, x, y, z) * dt
    k_v3 = dvdt(v_1 , v_2+ k_v2/2, v_3, x, y, z) * dt
    k_v4 = dvdt(v_1 , v_2+ k_v3, v_3, x, y, z) * dt

    k_y1 = v_2 * dt
    k_y2 = (v_2 + k_v1/2) * dt
    k_y3 = (v_2 + k_v2/2) * dt
    k_y4 = (v_2 + k_v3) * dt

    return (k_y1 + 2 * k_y2 + 2 * k_y3 + k_y4) / 6 , (k_v1 + 2 * k_v2 + 2 * k_v3 + k_v4)/ 6

def z_rk4(v_1, v_2, v_3, x, y, z, dt):

    def dvdt(v_1, v_2, v_3, x, y, z ):
        c_drag = - k * d * rho(y)/m
        speed = np.sqrt(v_1**2 + v_2**2 + v_3**2)

        dv_cor  = 2 * omga*(v_1 * np.cos(theta) + v_2 * np.sin(theta))

```

```

    dv_drag = c_drag * speed * v_3
    dv_cf = omga**2 * z

    return dv_cor + dv_drag + dv_cf

k_v1 = dvdt(v_1, v_2, v_3, x, y, z) * dt
k_v2 = dvdt(v_1, v_2, v_3+ k_v1/2, x, y, z) * dt
k_v3 = dvdt(v_1, v_2, v_3+ k_v2/2, x, y, z) * dt
k_v4 = dvdt(v_1, v_2, v_3+ k_v3, x, y, z) * dt

k_z1 = v_3 * dt
k_z2 = (v_3 + k_v1/2) * dt
k_z3 = (v_3 + k_v2/2) * dt
k_z4 = (v_3 + k_v3) * dt

return (k_z1 + 2 * k_z2 + 2 * k_z3 + k_z4) / 6, (k_v1 + 2 * k_v2 + 2 * k_v3 + k_v4) / 6

# Solution of the simulation
def numerical_Euler(x, y, z, dt, t):

    x_numerical = [x]
    y_numerical = [y]
    z_numerical = [z]
    time_vlaues_numerical = [t]
    v_x = v_x0
    v_y = v_y0
    v_z = v_z0

    # # the loop for the numerical solution
    while y >= 0:
        dv_x = v_xe(v_x, v_y, v_z, x,y,z, dt)
        dv_y = v_ye(v_x, v_y, v_z, x,y,z, dt)
        dv_z = v_ze(v_x, v_y, v_z, x,y,z, dt)

        x += v_x * dt
        y += v_y * dt
        z += v_z * dt

        v_x += dv_x
        v_y += dv_y
        v_z += dv_z

        t += dt
        x_numerical.append(x)
        y_numerical.append(y)
        z_numerical.append(z)
        time_vlaues_numerical.append(t)

    return x_numerical, y_numerical, z_numerical, time_vlaues_numerical

def numerical_RK4(x, y, z, dt, t):
    x_numerical = [x]
    y_numerical = [y]
    z_numerical = [z]
    time_vlaues_numerical = [t]
    v_x = v_x0
    v_y = v_y0

```

```

v_z = v_z0

while y >= 0:
    dx , dvx = x_rk4(v_x, v_y, v_z, x, y, z, dt)
    dy , dvy = y_rk4(v_x, v_y, v_z, x, y, z, dt)
    dz , dvz = z_rk4(v_x, v_y, v_z, x, y, z, dt)

    x += dx
    y += dy
    z += dz

    v_x += dvx
    v_y += dvy
    v_z += dvz

    t += dt
    x_numerical.append(x)
    y_numerical.append(y)
    z_numerical.append(z)
    time_vlaues_numerical.append(t)

return x_numerical, y_numerical, z_numerical, time_vlaues_numerical

def main():

    dt = config.dt
    t = config.t_0
    x = 0
    y = 0
    z = 0

    x_Euler, y_Euler, z_Euler, time_vlaues_Euler = numerical_Euler(x, y, z, dt, t)
    x_RK4, y_RK4, z_RK4, time_vlaues_RK4 = numerical_RK4(x, y, z, dt, t)

    Stitle = "Solutions in x, y, z (m) vs time (s)"

    fig, axs = plt.subplots(3, 1, figsize=(14, 10))
    fig.suptitle(Stitle, fontsize=16)
    plt.subplots_adjust(hspace=0.5)

    add_figure(axs ,x_Euler, y_Euler, z_Euler, time_vlaues_Euler, "Numerical Solution(Euler)", 'k')
    add_figure(axs ,x_RK4, y_RK4, z_RK4, time_vlaues_RK4, "Numerical Solution(RK4)", 'r')
    plt.show()

    print_data(x_Euler, y_Euler, z_Euler, time_vlaues_Euler, "Numerical_Solution(Euler).xlsx")
    print_data(x_RK4, y_RK4, z_RK4, time_vlaues_RK4, "Numerical_Solution(RK4).xlsx")

```

Here's the code in config.py:

```

import numpy as np
# Mass of the shell 106 kg
# Muzzle velocity 1640 m/s
# Maximum firing rage 130 km
# Elevation 55°
# Maximum altitude 42.3 km
# Diameter of the shell 211 mm
# Drag coefficient* 0.1

m = 106

```

```

v_muzzle = 1640
r_max = 130000
theta = 41 * np.pi/180
h_max = 42300
d = 0.211
k = 0.1

v_x0 = 0
v_y0 = v_muzzle * np.sin(55 * np.pi/180)
v_z0 = v_muzzle * np.cos(55 * np.pi/180)
g = 9.81
omga = 7.3e-5

# time step
dt = 0.1
t_0 = 0

```

Here's the code in SolveODE.py:

```

import numpy as np
import matplotlib.pyplot as plt
import config
# from Main import *
from scipy.integrate import solve_ivp
from matplotlib.animation import FuncAnimation
from mpl_toolkits.mplot3d import Axes3D
import pandas as pd
from scipy import stats

# assume that the ODE is in the form of  $dy/dx = f(x, y)$ 
# and  $F_{\text{drag}} = -\kappa \rho A v^2$ 

#####
# plot the figure

def plot_figure(x,y,z,t,title,C):
    x = np.array(x)
    y = np.array(y)
    z = np.array(z)
    t = np.array(t)
    # only draw the figure with  $y \geq 0$ 
    mask = y >= 0
    t = t[mask]
    x = x[mask]
    y = y[mask]
    z = z[mask]

    # plot data
    # Size of figure in inches
    # (This may show up differently in the Jupyter Notebook but will
    be respected when you save the figure.)
    xSize = 6
    ySize = 3
    # Fontsize of various text elements of the figure in pts
    fontSize = 14
    # Size of line of the graph
    markerSize = 12
    # Color of line: 'r'=red, 'k'=black, 'b'=blue, 'g'=green, etc

```

```

markerColor = C
# Tick labels size
tickSize = 12

Stitle = title

fig, axs = plt.subplots(3, 1, figsize=(14, 10))
fig.suptitle(Stitle, fontsize=16)
plt.subplots_adjust(hspace=0.5)

# the figure of x
axs[0].set_title("x(t)", fontsize=fontSize)
axs[0].scatter(t,x, color=markerColor, s=markerSize)
axs[0].plot(t,x, color=markerColor, linewidth=2)
axs[0].set_ylabel("x(m)", fontsize=fontSize)
axs[0].set_xlabel("t(s)", fontsize=fontSize)

axs[1].set_title("y(t)", fontsize=fontSize)
axs[1].scatter(t,y, color=markerColor, s=markerSize)
axs[1].plot(t,y, color=markerColor, linewidth=2)
axs[1].set_ylabel("y(m)", fontsize=fontSize)
axs[1].set_xlabel("t(s)", fontsize=fontSize)

axs[2].set_title("z(t)", fontsize=fontSize)
axs[2].scatter(t,z, color=markerColor, s=markerSize)
axs[2].plot(t,z, color=markerColor, linewidth=2)
axs[2].set_ylabel("z(m)", fontsize=fontSize)
axs[2].set_xlabel("t(s)", fontsize=fontSize)

plt.show()

def add_figure(axs, x, y, z, t, label, color):
    x = np.array(x)
    y = np.array(y)
    z = np.array(z)
    t = np.array(t)
    # only draw the figure with y >= 0
    mask = y >= 0
    t = t[mask]
    x = x[mask]
    y = y[mask]
    z = z[mask]

    # plot data
    markerColor = color

    fontSize = 14

    # the figure of x
    axs[0].set_title("x(t)", fontsize=fontSize)
    axs[0].plot(t, x, color=markerColor, linewidth=2, label=label)
    axs[0].set_ylabel("x(m)", fontsize=fontSize)
    axs[0].set_xlabel("t(s)", fontsize=fontSize)
    axs[0].legend()

    axs[1].set_title("y(t)", fontsize=fontSize)
    axs[1].plot(t, y, color=markerColor, linewidth=2, label=label)

```

```

    axs[1].set_ylabel("y(m)", fontsize=fontSize)
    axs[1].set_xlabel("t(s)", fontsize=fontSize)
    axs[1].legend()

    axs[2].set_title("z(t)", fontsize=fontSize)
    axs[2].plot(t, z, color=markerColor, linewidth=2, label=label)
    axs[2].set_ylabel("z(m)", fontsize=fontSize)
    axs[2].set_xlabel("t(s)", fontsize=fontSize)
    axs[2].legend()

def print_data(x, y, z, t, filename):
    # convert the data to DataFrame
    data = {
        't': t,
        'x': x,
        'y': y,
        'z': z
    }
    df = pd.DataFrame(data)

    # DataFrame write in Excel file
    df.to_excel(filename, index=False)
    print(f"the data has been saved to{filename}")

def main():
    # test code

if __name__ == "__main__":
    main()

```